

033 动量学习率与调度练习详解

以下答案与正文、脚本和Notebook使用同一参数约定。动量缓冲为梯度单位，先加权旧缓冲与当前梯度，再乘本次学习率。第4题中的输入偏置仍写b，但手算时用u表示动量缓冲，以免混淆。

1 从递推得到权重和 方差和滞后

由 $b_k = \mu b_{k-1} + g_k$ 、 $b_{-1} = 0$ ，先得到 $b_0 = g_0$ 、 $b_1 = \mu g_0 + g_1$ 、 $b_2 = \mu^2 g_0 + \mu g_1 + g_2$ 。每次代入前一项都会给旧项再乘一个 μ ，所以第j步前的梯度权重为 μ^j 。

归一化EMA是 $m_k = (1 - \mu)b_k$ 。有限权重和为

$$S_k = (1 - \mu) \frac{1 - \mu^{k+1}}{1 - \mu} = 1 - \mu^{k+1}.$$

例如 $\mu = 0.8$ 、梯度序列[1,-1,1]：缓冲依次为1、-0.2、0.84；EMA为0.2、-0.04、0.168。第三个EMA的展开为 $0.128(1) + 0.16(-1) + 0.2(1) = 0.168$ ，权重和为0.488。若这三个输入来自恒定均值分布，零初始化均值修正为 $0.168/0.488 \approx 0.3442623$ 。这不是在宣称当前真实梯度就是该数；修正只针对指定均值模型。

方差推导要平方权重，并利用不同步噪声协方差为0： $\text{Var}(m_k) = \sigma^2(1 - \mu)^2 \sum_j \mu^{2j}$ 。不能把权重和直接当作方差因子。代入有限几何级数后得到正文公式；令k趋近无穷，得到 $\sigma^2(1 - \mu)/(1 + \mu)$ 。若相邻噪声正相关，还存在正交叉项，平滑收益可能小于这个理想值。

梯度突变为相反方向时，旧方向的总贡献不会瞬间消失，所以EMA会滞后。动量可能继续跨过最低点，这与它能平滑交替方向并不矛盾。图2的右面板是这个取舍的直接示意。

2 狭长谷地的稳定条件

SGD沿曲率 λ 的方向满足 $q_{k+1} = (1 - \alpha\lambda)q_k$ 。绝对值小于1才会缩小，因此 $0 < \alpha\lambda < 2$ 。 $\lambda=40$ 、 $\alpha=0.045$ 时系数-0.8，符号翻转且幅度缩小； $\lambda=1$ 时系数0.955，幅度慢慢缩小。 $\alpha=0.06$ 使陡方向系数-1.4，误差扩大。

固定步长动量消去缓冲后为 $q_{k+1} = (1 + \mu - \alpha\lambda)q_k - \mu q_{k-1}$ 。令 $a = \alpha\lambda$ ，特征多项式是 $p(r) = r^2 - (1 + \mu - a)r + \mu$ 。两个根的乘积是 μ 。

当 $0 \leq \mu < 1$ 时，若两根是共轭复根，它们的模都为 $\sqrt{\mu} < 1$ 。若是实根，满足 $p(1) = a > 0$ 、 $p(-1) = 2 + 2\mu - a > 0$ ，并且根乘积小于1，就排除了某个实根越出[-1,1]：跨出一边会让该边的多项式值非正，若两个根同时越出同一边则乘积至少1。反过来，两个根都严格在单位圆内时，上述三个不等式必须成立。因此稳定区间是 $0 < a < 2(1 + \mu)$ 。

本例 $\mu=0.8$ 时右端为3.6。 $\alpha=0.045$ 、 $\lambda=40$ 给出 $a=1.8$ ，特征式为 $r^2 + 0.8 = 0$ ，两个根为纯虚共轭根，模约0.8944； $\lambda=1$ 给出 $r^2 - 1.755r + 0.8 = 0$ ，也是模约0.8944的共轭根。这个结果解释了振荡收敛，不表示函数值每步单调，也不为改变LR后的非定常递推提供通用保证。

3 两种动量约定在调度下不同

设 $g_0 = 2, g_1 = 1, \mu = 0.8, \alpha_0 = 0.1, \alpha_1 = 0.05$ 。按本讲梯度缓冲： $b_0 = 2, b_1 = 0.8(2) + 1 = 2.6$ ，两次向负方向位移分别为0.2与0.13。

若直接使用位移缓冲 $v_k = \mu v_{k-1} + \alpha_k g_k$ ，则 $v_0 = 0.2, v_1 = 0.8(0.2) + 0.05(1) = 0.21$ 。第二次位移不同，差异不是浮点误差。

正确换元 $d_k = \alpha_k b_k$ 后，旧位移的系数应为 $\mu \alpha_k / \alpha_{k-1}$ 。第二步因此是 $0.8(0.05/0.1)(0.2) + 0.05 = 0.13$ 。当学习率恒定，比值等于1，两种写法才可用这一换元相互对应。前一步学习率为0时不能做除法，应回到原始缓冲递推。

4 两次完整链式计算的核对表

原始前向 $Z = [[1.2, 0.8], [-0.1, 0.8]]$ ， $H = [[1.2, 0.8], [0, 0.8]]$ ，预测[0.42, -0.3]、残差[-0.28, -0.1]，loss为0.0221。均值损失对两预测的导数为[-0.14, -0.05]。

乘旧输出权重[0.6, -0.5]后，隐藏梯度两行为[-0.084, 0.07]和[-0.03, 0.025]；乘ReLU门后，第二行第一列变成0。所有网络局部导数、含shape的张量和输入梯度均在trace.json中，下面重点把共享参数的两条样本路径列齐。

参数	样本1贡献	样本2贡献	合计
W11	-0.084	0	-0.084
W12	0.070	-0.025	0.045
W21	-0.168	0	-0.168
W22	0.140	0.025	0.165
b1	-0.084	0	-0.084
b2	0.070	0.025	0.095
v1	-0.168	0	-0.168
v2	-0.112	-0.040	-0.152
c	-0.140	-0.050	-0.190

W12的第一条路径是 $(1/2)(-0.28)(-0.5)(1)(1) = 0.07$ ，第二条是 $(1/2)(-0.1)(-0.5)(1)(-1) = -0.025$ 。其中五个因子依次来自均值、残差、输出权重、ReLU门和输入坐标。路径相加得到0.045。W21第一条路径最后一个因子为输入第二坐标2，故贡献-0.168；第二条因ReLU门为0而消失。

u初始为0，故第一次u就是合计梯度。学习率0.1后，新参数顺序为 [0.5084, -0.2045, 0.3168, 0.3835, 0.1084, 0.1905, 0.6168, -0.4848, 0.119]。重新前向得到预测 [0.52519232, -0.2584168]，loss约0.0084925618773056。

第二次预测梯度为[-0.08740384, -0.0292084]，新的隐藏梯度来自乘[0.6168, -0.4848]。ReLU门不变，参数路径贡献为：

参数	样本1贡献	样本2贡献	新梯度
W11	-0.053910688512	0	-0.053910688512
W12	0.042373381632	-0.014160232320	0.028213149312
W21	-0.107821377024	0	-0.107821377024
W22	0.084746763264	0.014160232320	0.098906995584
b1	-0.053910688512	0	-0.053910688512
b2	0.042373381632	0.014160232320	0.056533613952
v1	-0.109289761536	0	-0.109289761536
v2	-0.065815091520	-0.022738739400	-0.088553830920
c	-0.087403840000	-0.029208400000	-0.116612240000

对W12，旧缓冲0.045乘0.8得0.036，加新梯度得0.064213149312，乘-0.05得位移-0.0032106574656。因此新权重为-0.2077106574656。对所有参数同时做同样操作，得到正文第二次更新表；新预测约[0.6002412459735114, -0.2297098859883156]，新loss约0.00270862158258904。

若算出W12梯度0.0225，通常是均值因子重复除以2。若第二次仍使用0.045，多半重复了旧梯度。若把0.8乘在参数而非缓冲上，则算法本身已变。若先更新v再用新v计算同一次W梯度，则混合了不同参数点的导数。

5 50样本时怎样处理尾部

48样本、microbatch4、累积2时，每epoch是12个微批次、6次参数更新；12epoch是72次更新、576次样本呈现。预热12次更新对应2个epoch，并非12个epoch。

若改成50样本、保留所有样本且每epoch结束时清空累积窗口，前12个微批次各4样本，第13个微批次为2样本。会有6个8样本窗口和1个2样本窗口，每epoch7次更新，12epoch84次，样本呈现600次。末窗口损失应为这2个样本平方误差之和除以 2×2 ；不能仍除以 2×8 ，否则最后一步梯度被缩小到四分之一。

当前教学函数明确要求完整等大小窗口，因而拒绝50样本配置。这个拒绝避免用错分母，而不是说真实训练不能有尾批。如果选择drop_last或跨epoch继续累积，则样本覆盖、更新数或保存边界也会改变，必须重新声明协议和恢复状态。

图7的错误曲线把调度索引变成2k：原本12更新的预热只在约6更新内完成，原72更新的衰减在36更新附近完成。只修改累积次数而保留旧的“每batch调度”位置，是常见单位错误。

6 恢复需要什么 第一眼为什么会被骗

原定72更新，已完成24更新，就剩48更新。至少恢复参数、动量缓冲、optimizer各组当前LR及配置、scheduler计数和函数配置，以及决定后续批次的随机状态。还要确认模型结构、参数顺序、数据、版本和保存边界与原实验一致。

本讲的完整检查点在epoch4结束且scheduler.step完成后保存，下一次更新为k=24，LR约0.05398294095933821。重建LambdaLR时初始化计数为0，然后才载入原scheduler和optimizer状态。函数本身没有被state_dict保存，必须使用原实现和原总预算T=72。

故意漏scheduler时，optimizer仍带回正确当前LR，因此恢复后的第一步看起来正常；该步之后新scheduler只前进到1，于是下一步用0.01。完整恢复下一步应为约0.05306491028788964。检查连续几个实际使用LR，就能发现“只有加载瞬间正确”的假象。

仅比较loss不充分：漏RNG的最后loss约0.00091456，甚至略低于完整恢复的0.00091694，但样本序列、参数和随机状态已经不同。正确验收是完整状态与全部逐步记录相同。本讲采用同一运行版本下最终JSON字节一致作为更严格证据。

保存点若在累积窗口中途，还需未应用的梯度、窗口已消费的样本、微批次游标等；若有Dropout，还需相应随机源；若多worker预取，还需处理采样与队列状态。当前实验没有这些组件，不能把完整epoch边界的成功扩大成任意时刻恢复保证。

7 如何解释四方案结果

此例固定动量最终half-MSE约0.00047501，低于另三种预定配置。可以说“该CPU合成任务、该初始化、该固定预算下，固定动量拟合更充分”。不能说“固定动量普遍最好”“warmup无用”或“预热一定必要”。二次函数上的终点排序也不必与小网络相同，因为几何、梯度路径和预算均不同。

四方案等更新数、等样本呈现、同初值和顺序，让机制比较更容易解释，却没有消除超参数选择偏差。真正推荐方法时还需要合理且公平的调参预算、独立验证和跨种子不确定性。此处没有运行GPU速度或真实任务测试，也没有把CPU耗时差包装成硬件加速结论。